

Programming Assignment 8: Binary Search Trees (BSTs)

Due: Thursday, April 9th at 11pm CT

Learning Goals

- To implement a BST and its core operations
- To analyze the behavior of BSTs when adding randomized data versus sorted data
- To implement methods based on documentation in starter code, rather than on assignment handouts

Assignment Guidelines and Specifications

This is an individual assignment, which means you cannot work with a partner on this assignment. All code submitted must be entirely your own.

Getting Started

Here are the necessary files for this assignment:

- [BinarySearchTree.java](#): This file contains the starter code for the BST class. You will need to complete this file according to the given specifications and submit this file.
- [BSTTester.java](#): This file contains the main method used for testing and the provided correctness tests. You will need to write your own tests in this file, write the answers to your experiments in this file, and then submit this file.
- [Stopwatch.java](#): This file contains the Stopwatch class, which you will need to use in your experiments. You will use this class to measure the time elapsed. You should not edit this code and do not need to submit this file.

BinarySearchTree

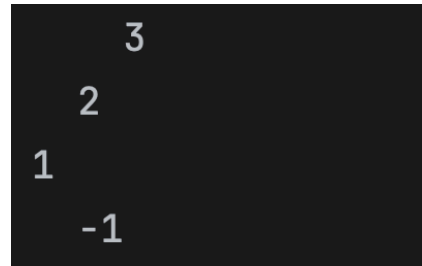
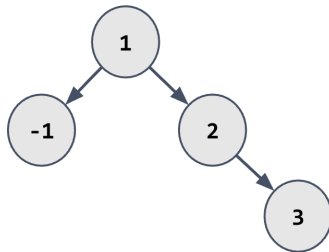
In this assignment, you will implement your own generic binary search tree class! You will need to determine what instance variables and constructors should be included in this class. You will be writing 13 different methods for this class. Rather than explaining all 13 methods in the assignment handout, we expect that you will write the methods directly based on the documentation (comments) already present in the starter code.

We have already written one of the methods together in lecture and have released videos on some of the other methods. You may, and are encouraged to, use this code on your assignment. **If you do so, you must cite in a comment that the code is from lecture.** For all the other methods, **you must design and implement the algorithms for this assignment on your own** (along with appropriate guidance from the teaching team during help hours and on Ed). Do not look up algorithms on the internet or from a textbook, especially not for balanced binary search trees. As always, for your methods, you may (and are encouraged to) create private helper methods as necessary in your solution.

The `get(int kth)`, `getAllLessThan(E val)`, and `getAllGreaterThan(E val)` methods should be as time- and space-efficient as possible. This means that the $T(N)$ for all these methods should be as small as possible. Here are some hints for these methods:

- Stop your search or processing as soon as you can. Don't do any extra work.
- Do **not** use the `getAll` method within these methods, because you will very likely fail some stress tests.
- All of these methods **must** be solved with recursive backtracking.

Additionally, in the starter code, there is a `printTree` method that prints out the representation of the current tree. Use this method when debugging your code. Note that this method prints the tree so that the root is to the left (rather than at the top). In other words, the trees are rotated 90 degrees counterclockwise. Read the documentation of this method for more information about its output. Here is an example of a BST (on the left) and what the `printTree` output would look like for the tree (on the right):



Note that you may not use the `TreeSet` or `TreeMap` classes from the Java standard library when implementing your BST. (You will, however, use the Java `TreeSet` class later in your experiments.)

Testing

You are required to write and submit your own test cases in `BSTTester.java`. You are expected to write 2 new test cases for each of the 13 methods, for a total of 26 test cases.

In previous assignment guides, we shared a few notes with you about writing test cases. Those notes are copied here for your convenience:

- You may copy the structure of the provided tests.
- Any test cases that you write should not violate the preconditions of the method.
- You must group the test cases by method and comment which method the test case is for.
- You are not required to write test cases for any private helper methods or constructors.
- **Delete the test cases that we have provided for you.**

Experiments

Now it's time to use your completed class to perform some experiments! You will use the Stopwatch class to record the time it takes to perform various operations on your BinarySearchTree class and the Java TreeSet class.

Write your results and answers in a comment at the top of the BSTTester . java file. Include the code to conduct the experiments in the BSTTester . java file, but before submitting, make sure to comment out all experiment code (especially any code that utilizes the Stopwatch class).

Note: You may not share your experiment code with other students. This is still code that you are expected to write individually and still follows the academic integrity guidelines set forth.

Experiment 1: Randomized Data

1. Create an empty BST and insert 1000 randomly generated Integer objects using the Random class. Here is one way you might choose to do this:

```
int n = 1000;
Random r = new Random();
BinarySearchTree b = new BinarySearchTree();
for(int i = 0; i < n; i++) {
    b.add(new Integer(r.nextInt() ) );
}
```

2. Use the Stopwatch class to record the time it takes to do this. Additionally, record the height of the tree after inserting the 1000 items. Then, record the size of the tree (note that the size may not be 1000 because the random number generator may produce duplicate values).
3. Repeat this experiment 10 times and record the average amount of time to do the insertions, the average height of the tree, and the average size of the tree.
4. Repeat the experiment 10 times for the following values of N: 2,000, 4,000, 8,000, 16,000, 32,000, 64,000, 128,000, 256,000, 512,000, and 1,024,000. Record all the previously stated information (you can, of course, automate these experiments to save time).
5. Answer the following question: For each value of N, what is the minimum possible tree height, assuming N unique values are inserted into the tree? Why?
6. Repeat Steps 1 – 4 using the Java TreeSet class. For this part, you only need to note the average time for the insertions (you do not need to note the height or size of the tree).
7. Answer the following: How do the average times for TreeSet compare to your BinarySearchTree? What do you think is the cause for these similarities/differences?

Experiment 2: Sorted Data

1. Create an empty BST and insert 1000 Integer objects sorted in ascending order (i.e. 1, 2, 3, 4, ..., 1000)
2. Use the Stopwatch class to record the time it takes to do this. Additionally, record the height of the tree after inserting the 1000 items. Then, record the size of the tree.
3. Repeat this experiment 10 times and record the average amount of time to do the insertions, the average height of the tree (which you should be able to determine easily without calling the height method), and the average size of the tree (again, you should be able to determine easily without calling the size method).
4. Repeat the experiment 10 times for the following values of N: 2,000, 4,000, 8,000, 16,000, 32,000, and 64,000. Record all the previously stated information (you can, of course, automate these experiments to save time).
5. Predict how long it would take to add 128,000, 256,000, and 512,000 sorted integers into your binary search tree. Explain your prediction.
6. Repeat Steps 1 – 5 using the Java TreeSet class. For this part, you only need to note the average time for the insertions (you do not need to note the height or size of the tree).
7. Answer the following: How do these times compare to the times it took for you BinarySearchTree class when inserting integers in sorted order? What do you think is the cause for these differences?

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. Implement all the required methods?
2. Ensure your program passes the given tests? Note: passing the provided tests does not mean your solution is correct. We will use hidden tests to test edge cases.
3. Create your own additional test cases?
4. Conduct and record your answers for the experiments?
5. Answer the questions in the experiments section and include explanations as necessary?
6. Comment out your experiment code?
7. Fill in the header comments for both files?
8. Check the preconditions in public methods and throw exceptions as necessary?
9. Follow the [program hygiene guide](#)?
10. Follow the assignment specifications from page 3 of the [Assignment handout](#)?

If you have done all these things, then you're ready to submit BinarySearchTree.java and BSTTester.java to Gradescope!

Thank you to Professor Mike Scott for creating the previous version of this assignment!